Movie Ticket Booking App

Yozigo Full-Stack Intern Task: submission document

Candidate	Ashika Mishra
Email	ashikamishra63@gmail.com
Date	22 October 2026
GitHub	github.com/malicious28/yozigo-movie-booking
Live link	yozigo-movie-booking.onrender.com (free hosting: the first visit after a quiet period can take up to a minute)
Screen recording	loom.com/share/71e296c7996e49d886e282557c381f77
Stack	React + TypeScript, Node.js + Express + TypeScript, PostgreSQL

Contents

1. Summary
2. How to run it locally
3. Test accounts
4. Flows completed
5. Main decisions
6. How double booking is prevented
7. Database design
8. API overview
9. Tests
10. Assumptions and known limitations
11. Additions beyond the brief

1. Summary

This is a booking website for a cinema with one screen of 80 seats (rows A to H, seats 1 to 10). Customers can sign up, browse movies and shows, pick up to six seats, hold them for five minutes, pay through a simulated payment step, and see or cancel their bookings. Cinema staff can add and edit movies, schedule shows and see how each show is selling.

The main rule, that a seat can never be sold twice, is enforced by the backend and the database, not by the website. Automated tests send many booking requests for the same seat at the same moment and check that exactly one succeeds.

All ten flows in the brief work, including the three bonus flows. The sections below explain how to run the project, the decisions behind it and the database design.

2. How to run it locally

Requirements: Node.js 20 or newer, and Docker (used for the PostgreSQL database).

Option A: development mode

```
# 1. Start the database (PostgreSQL in Docker, on port 5434)
docker compose up -d db

# 2. Backend, on http://localhost:4000
cd backend
cp .env.example .env
npm install
npm run migrate
npm run seed        # demo accounts, movies, snacks and shows for the next 5 days
npm run dev

# 3. Frontend, on http://localhost:5173 (in a second terminal)
cd frontend
npm install
npm run dev
```

Then open `http://localhost:5173`. The frontend forwards `/api` requests to the backend, so both share one address.

Option B: everything in Docker

```
docker compose up --build -d
docker compose exec app node dist/scripts/seed.js
```

Then open `http://localhost:8080`. In this mode the backend also serves the built website.

Running the tests

```
docker compose up -d db
cd backend
npm test
```

The tests create and use their own database (`movie_booking_test`), so they do not affect the demo data. The database uses port 5434 so it does not clash with a PostgreSQL that may already be installed. Running the seed again is safe; it only adds shows that are missing.

3. Test accounts

Created by `npm run seed`. The admin account can only be created this way; normal sign-up always creates a customer.

Role	Email	Password
Admin	admin@cinema.test	admin12345
Customer	customer@cinema.test	customer123
Customer (second)	customer2@cinema.test	customer123

The second customer is useful for trying two people booking the same seat, in a normal and a private browser window.

4. Flows completed

#	Flow	Status	Notes
1	Sign up and log in	Done	Passwords hashed with bcrypt, at least 8 characters. Duplicate email and wrong login show clear messages; the login message does not say which part was wrong. Booking pages and APIs require a login.
2	Browse movies and shows	Done	Movies with poster, language, duration and rating. Shows grouped by date with time, price and seats left. Started shows are hidden.
3	Seat map	Done	8 x 10 map with a screen marker. Available, held and booked seats look different; held and booked seats cannot be clicked. Loaded from the backend.
4	Select and hold seats	Done	1 to 6 seats, selected seats and total shown. Hold lasts 5 minutes with a countdown. If a seat was taken meanwhile, the message names the seat and the map refreshes.
5	Pay and confirm	Done	Simulated payment with "Payment success" and "Payment failure". Failure keeps the hold so the user can retry. Paying after the hold expires is rejected. Double booking is prevented by the database and covered by tests.
6	My bookings	Done	Upcoming first, then past. Shows booking ID, movie, date, time, seats, amount and status. Other users' bookings are never visible.
7	Admin: movies and shows	Done	Add, edit and delete movies; add and delete shows. Overlapping shows are rejected. Shows with bookings cannot be deleted. Customers cannot use admin pages or APIs.
8	Bonus: cancel a booking	Done	Allowed up to 2 hours before the show. Seats become available again straight away.
9	Bonus: bookings per show	Done	Seats booked out of 80, percentage full, money collected, the list of bookings and the seat map.
10	Bonus: live seat updates	Done	The seat map refreshes every 3 seconds using polling. The reason for polling is given in section 5.

Nothing in flows 1 to 10 is known to be broken. Smaller limitations are listed in section 10.

5. Main decisions

1. **Node.js with Express and TypeScript** for the backend: one language across the whole project, and the same data types on both sides.
2. **PostgreSQL with plain SQL** instead of an ORM: the important logic here is locking and transactions, and writing the SQL directly keeps that logic visible and easy to check.

3. **One database row per seat per show**: each show gets 80 seat rows, which can be locked one by one while a hold or booking is made (section 6).
4. **Holds expire on their own**: a hold stores its end time and is ignored after that, so no background job is needed and an expired hold can never block a seat.
5. **A booking is created only when payment succeeds**, so a failed payment leaves nothing behind.
6. **The database enforces the key rules itself**: shows cannot overlap, booked shows cannot be deleted, and a seat can belong to only one booking.
7. **Login uses a JWT stored in an httpOnly cookie**: scripts in the page cannot read it. The user's role is checked against the database on every request.
8. **Live updates use polling every 3 seconds** rather than WebSockets: holds expire with time anyway, polling keeps working through server restarts, and an 80-seat map is a very small request.
9. **On the frontend, TanStack Query** handles loading, caching and refreshing data, and the active hold is read back from the server, so a page refresh does not lose it.

6. How double booking is prevented

Holding seats and confirming a booking each run inside a single database transaction with three steps:

1. **Lock** the requested seat rows (`SELECT ... FOR UPDATE`), always in the same order.
2. **Check** that every seat is still free (or, when paying, still held by this user and not expired).
3. **Update** the seats and, when paying, create the booking.

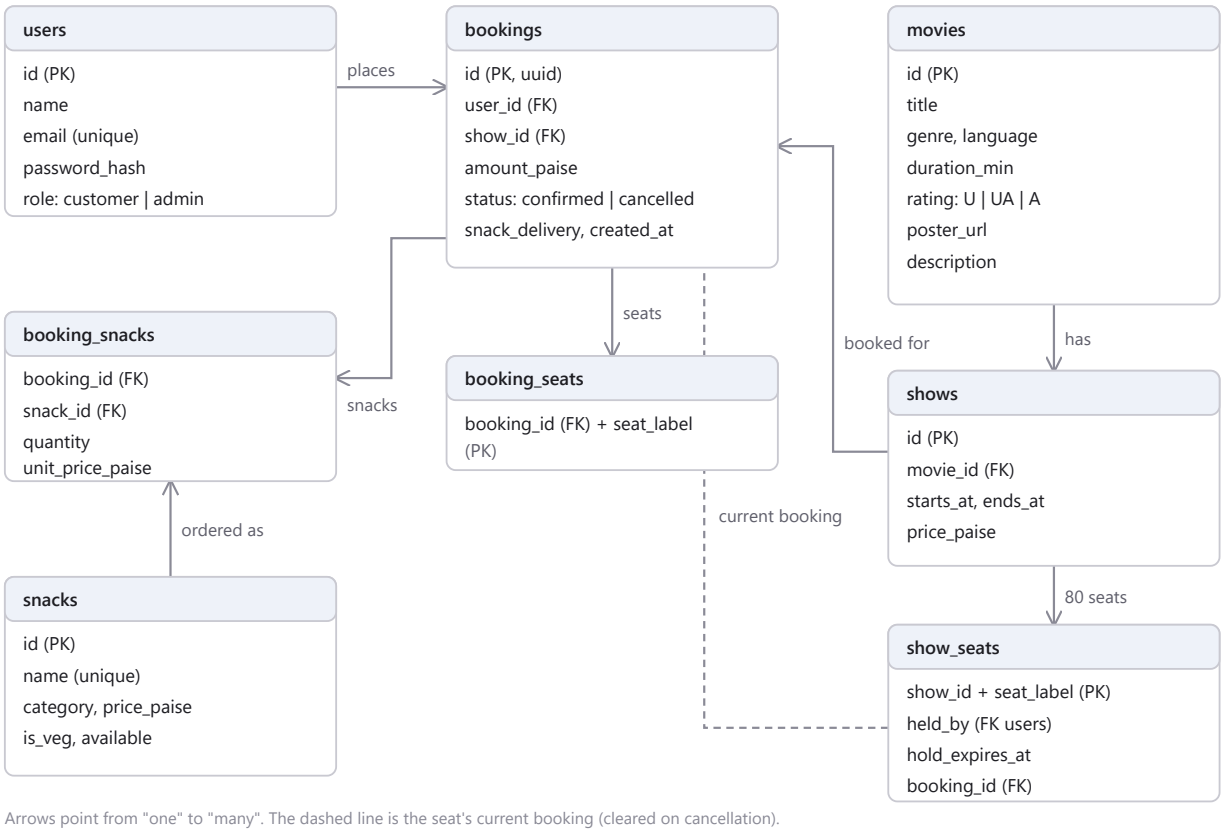
If two people ask for the same seat at the same moment, the database makes the second request wait until the first has finished. When the second request continues, it sees that the seat is now taken and returns an error naming the seat. Locking the seats in the same order every time stops two requests from waiting on each other forever.

As a second safeguard, the final update only changes seats that are still free, and the code checks how many rows actually changed. Each seat row also has a single booking column, so a seat can only ever point to one booking.

The tests prove this with real concurrent requests: two users racing for one seat, twenty users racing end to end, and the same payment sent five times at once. One test deliberately forces the two requests to overlap. With the locks removed, that test fails.

7. Database design

Eight tables. Money is stored in paise as whole numbers (for example, Rs 250 is stored as 25000) to avoid rounding errors. All times are stored with time zones.



Arrows point from "one" to "many". The dashed line is the seat's current booking (cleared on cancellation).

Table	Purpose	Rules kept by the database
users	Customer and admin accounts.	Email unique and stored in lower case; role is customer or admin.
movies	Title, genre, language, duration, rating, poster.	Duration above zero; rating is U, UA or A.
shows	A movie at a time and price. End time = start + duration.	No two shows may overlap in time (exclusion constraint).
show_seats	The 80 seats of each show and their current state: free, held (by whom, until when) or booked.	One row per seat per show; a seat cannot be held and booked at once.
bookings	A confirmed or cancelled booking.	A show with bookings, and its movie, cannot be deleted.
booking_seats	The seats in each booking, kept after cancellation.	Each seat listed once per booking.
snacks	The food and drinks menu.	Price above zero; fixed categories.
booking_snacks	Snacks ordered with a booking, with the price at the time.	Quantity from 1 to 10.

8. API overview

A REST API under `/api`. Errors always have the form `{ "error": { "code", "message" } }`.

Method	Path	Who can call it
POST	/auth/signup, /auth/login, /auth/logout	Anyone
GET	/auth/me	Logged-in user
GET	/movies, /movies/:id, /shows/:id, /snacks	Anyone
GET	/shows/:id/seats	Logged-in user
POST, DELETE	/shows/:id/hold	Logged-in user
GET, POST	/bookings (list; confirm after payment)	Logged-in user
GET	/bookings/:id	Owner of the booking only
POST	/bookings/:id/cancel	Owner of the booking only
POST, PUT, DELETE	/admin/movies, /admin/movies/:id	Admin only
GET, POST, DELETE	/admin/shows, /admin/shows/:id, /admin/shows/:id/bookings	Admin only

Asking for another user's booking returns "not found" rather than "forbidden", so the API does not reveal that the booking exists.

9. Tests

38 backend tests (Vitest and Supertest) run against a real PostgreSQL database, because the behaviour being tested (locks, constraints, transactions) only exists in the database.

File	What it covers
double-booking.test.ts	Two users holding the same seat at once; a forced overlap of two requests; twenty users racing end to end; the same payment sent five times; booking a seat someone else owns.
auth.test.ts	Password hashing, duplicate email, short password, same message for wrong email or password, logout, protected APIs rejecting missing or forged logins.
bookings.test.ts	Listing only future shows, holds, the 6-seat limit, replacing a hold, expiry, paying after expiry, snack prices taken from the server, privacy of bookings, ordering of My bookings, the 2-hour cancellation rule.
admin.test.ts	Customers blocked from admin APIs, movie add/edit/delete, show times in Indian time, overlap rules, delete rules, occupancy and revenue.

10. Assumptions and known limitations

Assumptions

- The cinema is in India, so show times are entered and displayed in Indian Standard Time, whatever the visitor's computer is set to.
- Shows that have started cannot be held or booked, as well as not being listed.
- Choosing different seats and pressing Continue again replaces the user's previous hold for that show.
- "Seats available" does not count seats currently held by someone else.
- Cancelled bookings still count as bookings when deciding whether a show can be deleted, so the history stays intact.

Known limitations

- A user can extend a hold by pressing Continue again before it runs out. A real system would cap the total hold time.
- There is no rate limiting on login yet. It would be the first thing to add before going live.
- Movie posters are linked from Wikipedia. If one cannot load, the app shows a generated title card instead.

11. Additions beyond the brief

- **Snacks delivered to the seat.** Food and drinks can be added during checkout, delivered at the start of the show or at the interval. Prices are always taken from the database.
- **E-ticket with a QR code** of the booking ID, shown after payment.
- **Checkout in three steps** (seats, snacks, pay) with an order summary that stays on screen, and quick switching between show times on the same day.
- **Show times coloured by availability** (available, filling fast, almost full), a seven-day date strip, search, and language and certificate filters.
- **Admin summary**: upcoming shows, seats sold, average occupancy and money collected.
- **Accessibility**: every seat is a real button that works with a keyboard and screen reader, seat states differ by shape as well as colour, and animations and transparency are reduced for users who ask for it.
- **Docker**: `docker compose up --build` runs the whole app on one address.

Credits

Demo movies are real releases. Their posters are linked from each film's English Wikipedia article and remain the copyright of their studios; they are used here only for demonstration. Smooth scrolling uses the open-source Lenis library.